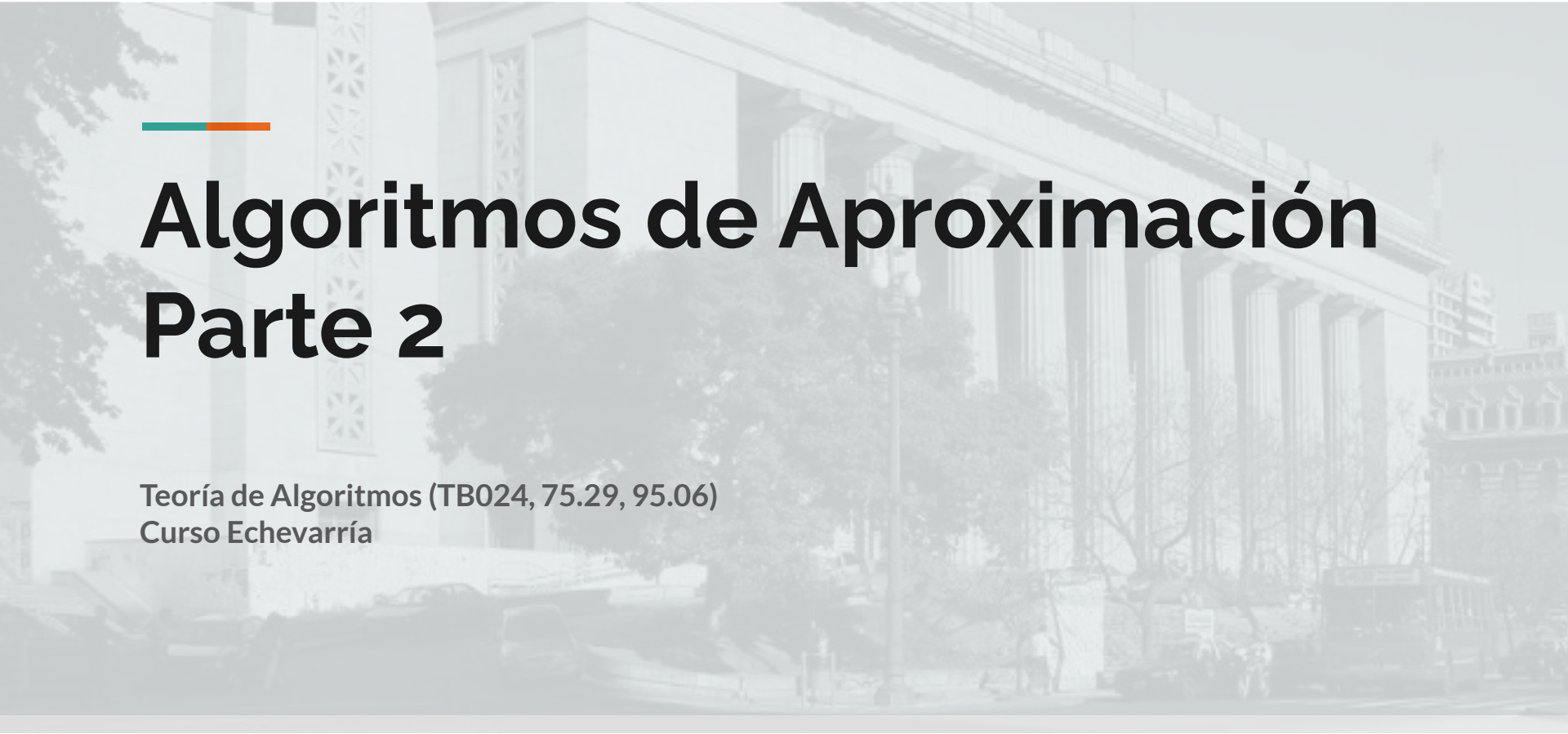
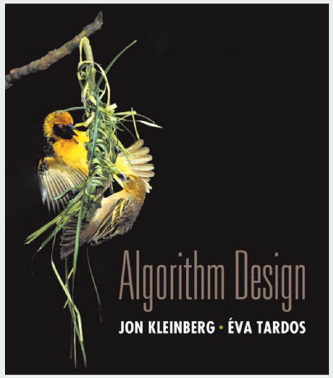


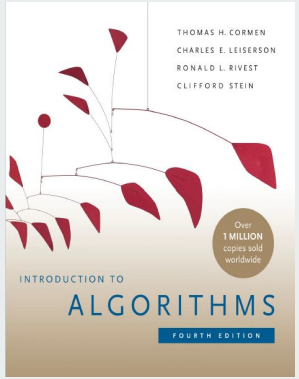
Algoritmos de Aproximación Parte 2

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso de la clase anterior



Cap 11



Cap 35



Algoritmos de Aproximación - Repaso

- En ocasiones, es preferible una buena solución rápida, que una solución óptima demasiado tarde.
- La primera idea para algoritmos aproximados, son los métodos Greedy.
- Es fundamental contar con una cota o límite para poder evaluar la solución obtenida (ya que no conocemos la solución óptima).
- Pricing methods (para grafos).
- Programación lineal: La Relajación lineal puede ser una buena cota.

Pricing Method



Pixabay

Pricing Method: Introducción

- También se lo conoce como Método Primal-Dual
- Fue perfeccionado por Dantzig, Ford y Fulkerson, allá por 1956 (*)
- Es un método general para desarrollar algoritmos de aproximación
  - Partimos de un grafo $G=(V,E)$, dirigido o no dirigido
  - Las aristas tienen asociado un costo
  - Se desea encontrar un subconjunto mínimo de aristas $S \subseteq E$ que cumplen con algún criterio dado
- Ejemplos:
  - Vertex Cover
  - Set Cover
  - Minimum Weight Perfect Matching, etc.

(*) G. B. Dantzig, L. R. Ford, and D. R. Fulkerson. A primal-dual algorithm for linear programs. In H. W. Kuhn and A. W. Tucker, editors, Linear Inequalities and Related Systems, pages 171–181. Princeton University Press, Princeton, NJ, 1956

Vertex Cover: Pricing Method

Vamos a aplicar Pricing Method para resolver aproximadamente a Vertex Cover

- Seguimos trabajando sobre la variante que añade costos a los vértices
- Seguimos buscando el conjunto de vértices que hace mínimo el peso total, cubriendo todas las aristas
- Imaginemos que cada arista tiene que *pagar* para ser cubierta por un vértice
  - ¿Qué tanto está dispuesta a pagar?
  - ¿Cuál es el precio acorde (fair)?
  - ¿Cuál es un precio excesivo (unfair)?
  - ¿Podrían aristas adyacentes a un vértice v_i pagar cada una una parte del costo de ser cubiertas por v_i?

Vertex Cover: Pricing Method (cont.)

- Vamos a tomar el peso w_i de seleccionar a v_i como el costo a pagar
- Cada arista será un agente dispuesto a pagar un precio al vértice que la cubre
- El algoritmo no sólo encontrará un Vertex Cover S, sino también determinará el precio $p_e \geq 0$ que pagará cada arista $e \in E$
- Esos precios p_e darán el costo total aproximado de S

Vertex Cover: Pricing Method (cont.)

- Definamos un precio acorde:
  - Seleccionar un vértice v_i cubre todas las aristas adyacentes
  - Sería excesivo pedirle a estas aristas que paguen más que w_i

Un precio acorde p_e debe cumplir que $\sum_{e=(i,j)} p_e \leq w_i \quad (1)$

- Esto significa que entre todas las aristas vecinas a un mismo vértice deben pagar a lo sumo el costo del vértice que las cubre, y no más

Vertex Cover: Pricing Method (cont.)

- Supongamos que tenemos una solución óptima S^* al problema de VC
- Sumemos todos los precios que se pagarían por todos los vértices de S^*:

$$\sum_{i \in S^*} \sum_{e=(i,j)} p_e \leq \sum_{i \in S^*} w_i = w(S^*) \quad (2)$$

- Como S* es un vertex cover, todas las aristas aportan al menos una vez su pago p_e al miembro izquierdo:
  - Al menos una vez al vértice que la cubrió
  - A lo sumo dos veces para el par de vértices (i,j) que la delimitan
- Como todos los $p_e \geq 0$, entonces la sumatoria del miembro izquierdo es al menos tan grande como la suma de todos los precios p_e:

$$\sum_{e \in E} p_e \leq \sum_{i \in S^*} \sum_{e=(i,j)} p_e \quad (3)$$

Vertex Cover: Pricing Method (cont.)

- Combinando (2) y (3) llegamos a:
- Y eso es una cota para la aproximación

$$\sum_{e \in E} p_e \leq w(S^*) \quad (4)$$

Vertex Cover: Algoritmo

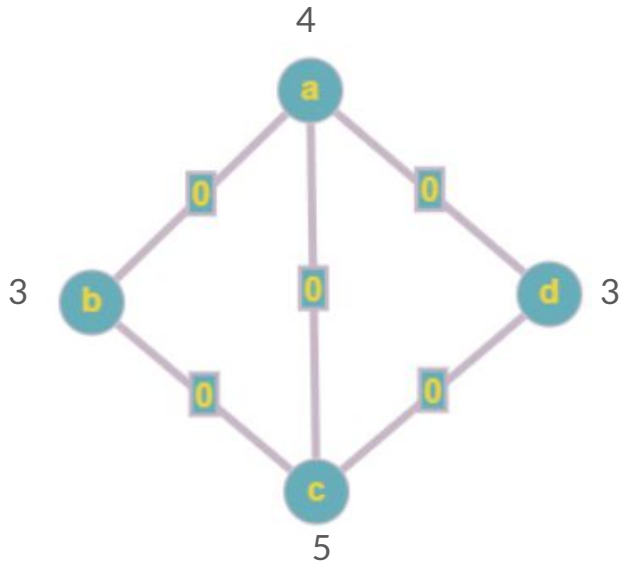
Proponemos un algoritmo Greedy

- Va a obtener tanto el S aproximado como los p_e a pagar por cada arista
- Definición: un vértice está pago sii $\sum_{e=(i,j)} p_e = w_i$
- Regla Greedy:
  - Tomar la próxima arista que no tenga pago ninguno de sus vértices (i, j)

Vertex Cover: Algoritmo (cont.)

```
def vc_aproximado(G, w):
  p(e)=0 para todas las aristas e en E
  mientras haya una arista e=(i,j) tal que ni i ni j
están pagos:
    seleccionar e
    incrementar p(e) sin violar el precio acorde
  S=conjunto de todos los vértices pagos
  devolver S
```

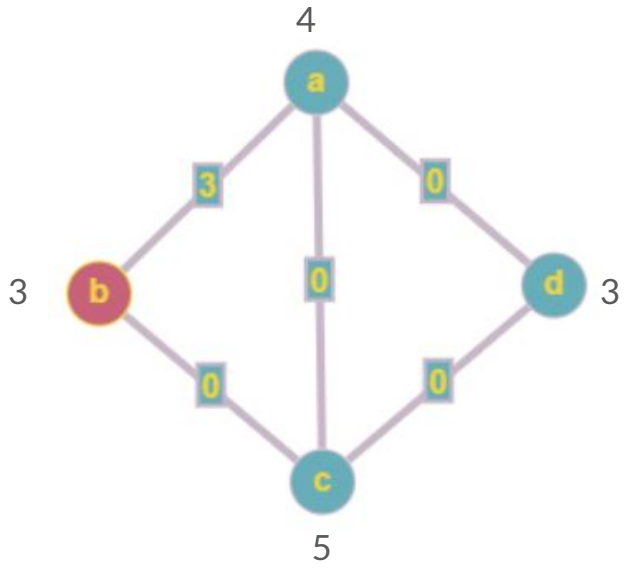
Vertex Cover: Algoritmo (cont.)



Ejemplo

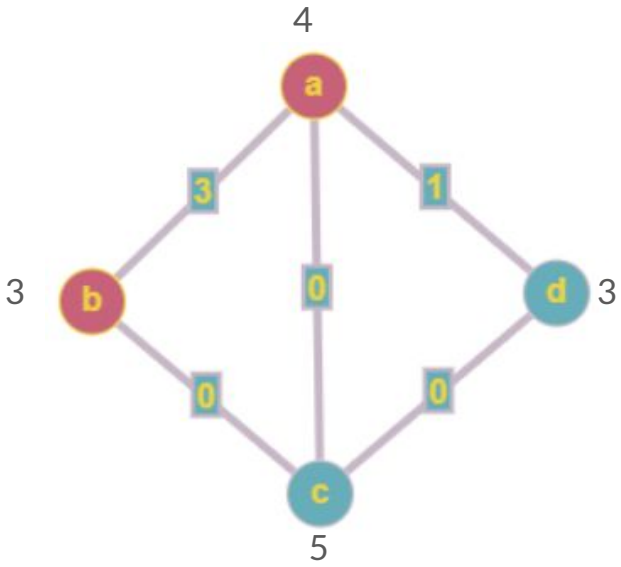
- Los costos w_i se indican al lado de cada vértice
- Al comienzo, todas las aristas tienen $p_e=0$ y ningún vértice está pago
- Empezamos con alguna arista para la cual ninguno de sus extremos estén pagos: elijamos (a,b)
- ¿Cuál es el mayor p_e acorde?
- El mayor p_e acorde es 3, ya que hace que el vértice b quede pago

Vertex Cover: Algoritmo (cont.)



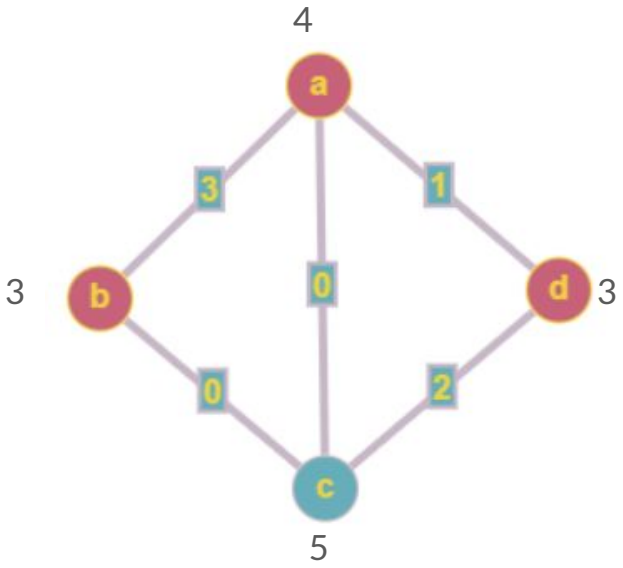
- Seguimos con otra arista para la cual ninguno de sus extremos estén pagos: elijamos (a,d)
- ¿Cuál es el mayor p_e acorde?
- El mayor p_e acorde es 1 porque a queda pago con los 3 que ya aportó (a,b)

Vertex Cover: Algoritmo (cont.)



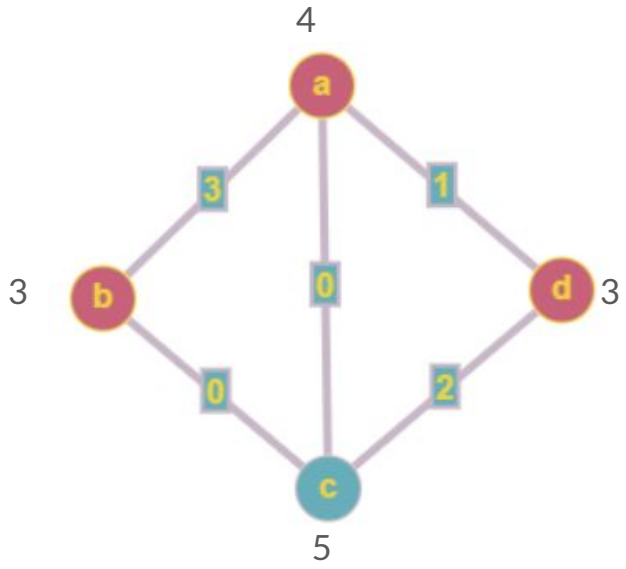
- Seguimos con otra arista para la cual ninguno de sus extremos estén pagos: elijamos (c,d)
- ¿Cuál es el mayor p_e acorde?
- El mayor p_e acorde es 2, ya que hace que el vértice d quede pago con el 1 que aporta (a,d)

Vertex Cover: Algoritmo (cont.)



- En este punto, se acabaron las aristas que no tienen ningún vértice pago
- Encontramos una solución:
  - $S=\{b, a, d\}$ (bueno, no es taaan mala)
- Encontramos los precios que debe pagar cada arista:
  - (b,a): 3
  - (a,d): 1
  - (d,c): 2
  - (c,b): 0
- El costo de S es $3+4+3=10$

Vertex Cover: Algoritmo (cont.)



Un momento…

- El costo de S es $3+4+3=10$
- ¿Pero lo que pagan las aristas es $3+1+2=6$?
- Bueno, sí… Pero en realidad (a,b) y (a,d) deberían pagar dos veces porque son cubiertas por sus dos vértices
- Ah! Entonces lo que pagan las aristas es $2*3+2*1+2 = 10$
- Entonces podemos afirmar que:

$$w(S) \leq 2 \sum_{e \in E} p_e \quad (5)$$

Vertex Cover: Conclusión

- Y juntando (4) con (5) nos queda que:

$$w(S) \leq 2 \sum_{e \in E} p_e \leq 2w(S^*)$$

- Nuestro algoritmo, en el peor de los casos, nos dará un Vertex Cover con un peso de hasta 2 veces el peso óptimo

Algoritmos arbitrariamente buenos: Otra mochila más



Algoritmos de Aproximación: Mochila

- El algoritmo que vamos a ver se aplica al problema estándar de la mochila: Tenemos que cargar cosas con un peso y un beneficio en una mochila que tiene un límite de peso aceptado, y buscando obtener el máximo beneficio posible.
- Además, debemos indicar un parámetro ε que indicará la distancia máxima a la que estará la solución obtenida.
- Para un épsilon determinado, el tiempo es polinomial.

Algoritmos de Aproximación: Mochila

- El algoritmo de PD para resolver el problema de la mochila que vimos en clase corre en $O(nW)$.
- Si la capacidad W y los pesos son pequeños, el algoritmo es rápido, aunque los beneficios sean grandes.
- Se puede obtener un algoritmo que sea rápido si los beneficios son pequeños, incluso si W y los pesos son grandes.
- El resultado es un algoritmo de complejidad temporal $O(n^2.v^*)$, siendo $v^* = max(v_i)$
- Esta estrategia se puede usar para cualquier algoritmo pseudopolinomial.

Algoritmos de Aproximación: Mochila

- Definimos $b = (e/2n)*Max\ (v_i)$
- A cada valor v_i lo redondeamos al múltiplo de b superior más cercano.
- Resolvemos el problema así reescalado por PD

- La solución obtenida es válida (no modificamos los w_i)
- El algoritmo es polinomial para un valor fijo de e.
- La única condición es que $1/e$ sea entero

Algoritmos de Aproximación: Mochila

- En el problema de la mochila, los subproblemas dependen de la capacidad disponible.
- Buscamos hacer que funcione bien para beneficios pequeños, independientemente de la capacidad.
- Vamos a guardar en $OPT\ (i,V)$ la solución usando los primeros i elementos y alcanzando el beneficio V, cuál es la capacidad más baja que lo logra.
- El tamaño de la matriz de memorización será a lo sumo $n^2.v^*$

Algoritmos de Aproximación: Mochila

¿Cuál es la complejidad temporal de este algoritmo?

- La PD tiene un $O(n^2v^*)$
- $v^* = max\ (v_i)$
- $b = (e(2n) * max\ (v_i))$
- Entonces, $max\ (v_i) = n\ e^{-1}$

- Entonces, **la complejidad del algoritmo es** $O(\ n^3\ e^{-1})$

- Al depender de e, sigue siendo pseudopolinomial.

Algoritmos de Aproximación: Mochila

- ¿Qué tan buena es la solución obtenida?
- Sea S^* cualquier solución válida; nuestro algoritmo encuentra la solución S que es óptima con los valores $\sim v_i$

$$\sum_{i \in S} \tilde{v}_i \geq \sum_{i \in S^*} \tilde{v}_i.$$

- Por lo tanto, se puede asegurar que:

$$\sum_{i \in S^*} v_i \leq \sum_{i \in S^*} \tilde{v}_i \leq \sum_{i \in S} \tilde{v}_i \leq \sum_{i \in S} (v_i + b) \leq nb + \sum_{i \in S} v_i,$$

- La solución que obtuvimos está, a lo sumo, (nb) por debajo de la óptima

Algoritmos de Aproximación: Mochila

- ¿Qué tan buena es la solución obtenida?

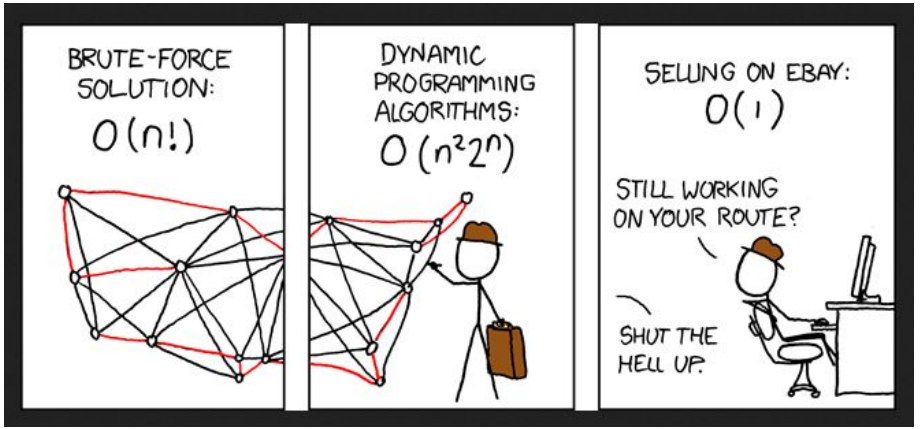
$$\sum_{i \in S^*} v_i \leq \sum_{i \in S^*} \tilde{v}_i \leq \sum_{i \in S} \tilde{v}_i \leq \sum_{i \in S} (v_i + b) \leq nb + \sum_{i \in S} v_i,$$

- La solución que obtuvimos está a lo sumo, (nb) por debajo de la óptima

$$\sum_{i \in S^*} v_i \leq \sum_{i \in S} v_i + nb \leq (1 + \epsilon) \sum_{i \in S} v_i.$$

- Quedando e como el factor de desvío de la solución obtenida respecto de la óptima

Algoritmos aproximados para el problema del viajante



http://imgs.xkcd.com/comics/travelling_salesman_problem.png

Algoritmos de Aproximación: Viajante

- Hay diferentes estrategias de algoritmos para enfrentar este problema
  - Greedy
  - Inserción
  - Árboles
- Dos formas de evaluarlas:
  - Garantía de calidad. (En general sólo para árboles)
  - Cota inferior (límite de Held-Karp: Relajación lineal)

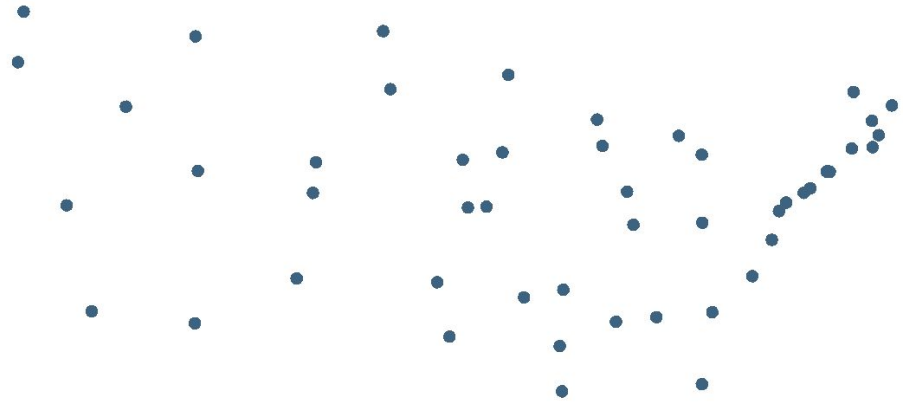


Algoritmos de Aproximación: Viajante

GREEDY

- TRAMO MÁS CORTO:

1. Ordenar los ejes de menor a mayor
2. Tomar el primer eje de la lista, siempre que sea “legal”:
    1. No cierre un ciclo con menos de N ejes
    2. No eleve el grado de ningún nodo a más de 2
3. Si el tour tiene menos de N nodos, volver a 2



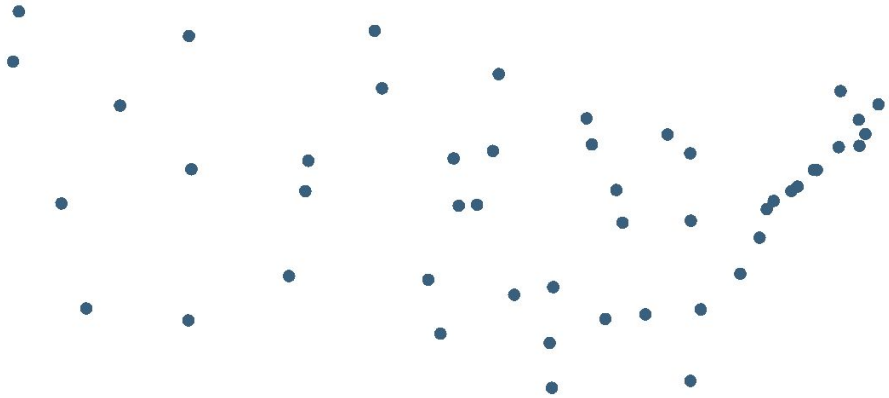
$O(n^2 log_2(n))$. No tiene garantía de calidad, pero en la práctica da resultados hasta un 15/20% por sobre el límite de Held-Karp

Algoritmos de Aproximación: Viajante

GREEDY

- VECINO MÁS CERCANO:

1. Comenzar por un nodo cualquiera
2. Buscar el nodo más cercano que aún no haya sido visitado e ir allí.
3. Si quedan nodos sin visitar, volver al paso 2.
4. Ir al nodo inicial.



O (n^2) No tiene garantía de calidad, pero en generalmente da valores hasta un 25% por sobre el límite de Held-Karp.

Algoritmos de Aproximación: Viajante

GREEDY

- BORUVKA:

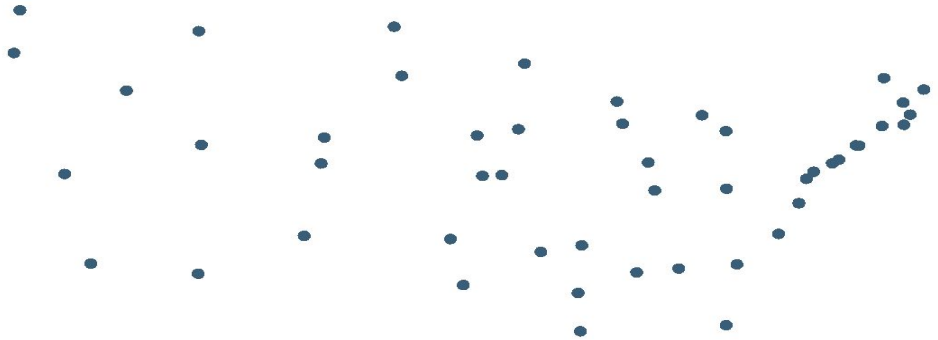
1. Elegir el vecino más cercano a cada nodo.
2. Ordenar los ejes seleccionados de menor a mayor.
3. Recorrer la lista e ir agregando todos los ejes “legales”.
4. Si no se formó un tour, volver al paso 1 considerando sólo los nodos con grado < 2.

Algoritmos de Aproximación: Viajante

INSERCIÓN

- INSERCIÓN MÁS CERCANA:

1. Seleccionar el eje más corto y crear un tour que lo recorra de ida y vuelta.
2. Seleccionar el nodo no visitado más cercano a un nodo visitado.
3. Buscar el eje del tour tal que el costo de intercalar la ciudad seleccionada en el lugar de dicho eje sea mínimo.
4. Si quedan nodos sin visitar, volver al paso 2.



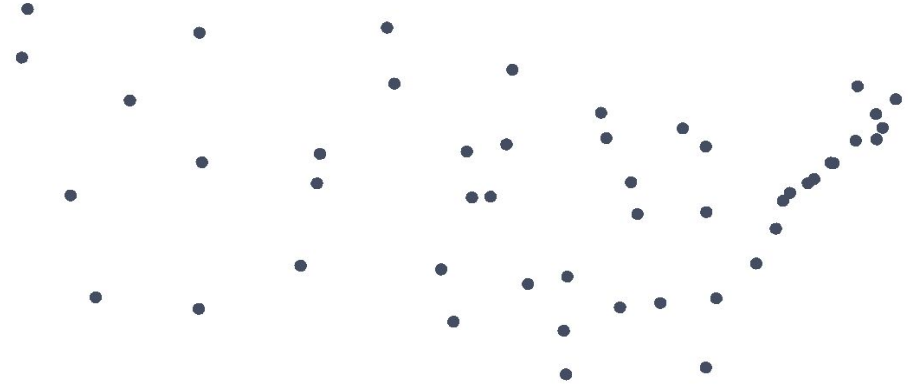
O (n^2) No tiene garantía de calidad.

Algoritmos de Aproximación: Viajante

- ALGORITMO DE CHRISTOFIDES:

1. Construir un árbol de cobertura mínimo que cubra todos los nodos.
2. Con los nodos que tengan orden impar en el árbol, construir un emparejamiento perfecto mínimo.
3. Con los ejes del árbol y el emparejamiento, construir un tour evitando los nodos ya visitados.

ÁRBOL





O (n^3) Este algoritmo tiene garantía de calidad 3/2. Aunque suele generar resultados mucho mejores, cerca de un 10% por sobre el límite de Held-Karp.

Algoritmos de construcción y de mejora.

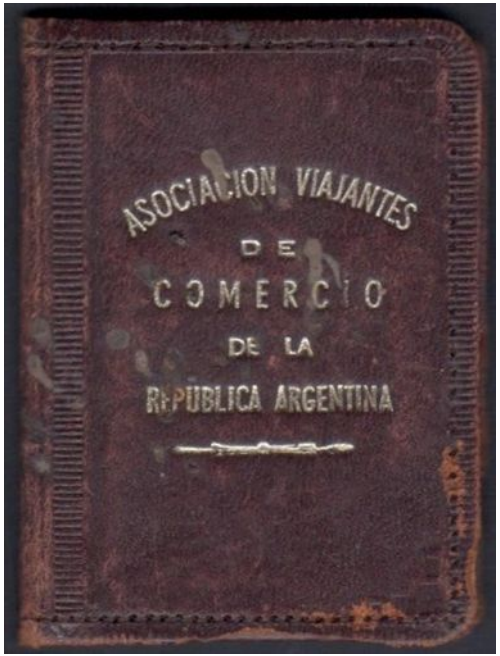


Algoritmos de Aproximación: Viajante

- COMPARACIÓN:

Heurística	Distancia a HK (%)	Tiempo (seg)
Christofides	6,7	14,1
Boruvka	15	0,26
Goloso	15,2	0,44
Inserción más cercana	22,8	2,84
Vecino más cercano	25,1	0,14

(Para el set de datos d15112)



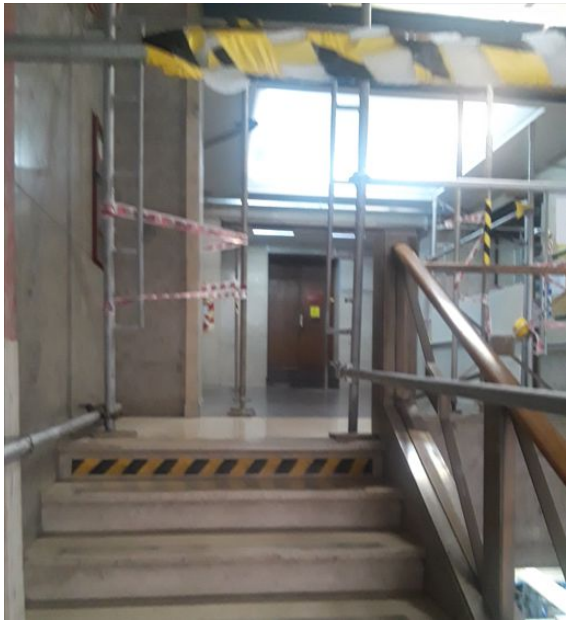
Algoritmos de Aproximación: Mejoramiento

- ALGORITMOS DE CONSTRUCCIÓN

  Construyen una solución a partir de los datos.

- ALGORITMOS DE MEJORAMIENTO

  Parten de una solución válida e intentan mejorarla.



Algoritmos de Aproximación: Mejoramiento

ALGORITMOS DE MEJORA PARA EL PROBLEMA DEL VIAJANTE

- **ALGORITMO de K-Intercambios:**
  - Dada una solución factible (un tour), tratar de mejorarla cambiando al mismo tiempo k ejes de la solución. (HAY QUE EXPLICITAR COMO SE ELIGEN LOS EJES A CAMBIAR)
  - Este procedimiento se basa en una propiedad de grafos euclídeos: “Si un ciclo Hamiltoniano se cruza a sí mismo, puede ser fácilmente acortado, basta con eliminar las dos aristas que se cruzan y reconectar los dos caminos resultantes mediante aristas que no se corten. El ciclo final es más corto que el inicial.”

Algoritmos de Aproximación: Mejoramiento

ALGORITMO de K-Intercambios:

- Sea S la solución actual
- Sea S' la solución obtenida al hacer un k-intercambio
- Si S' es mejor que S, definir $S = S'$
- Si no, determinar si existen otros $k\text{-}intercambios$ sin analizar. Si hay, ir a 1. Si no hay, finalizar.

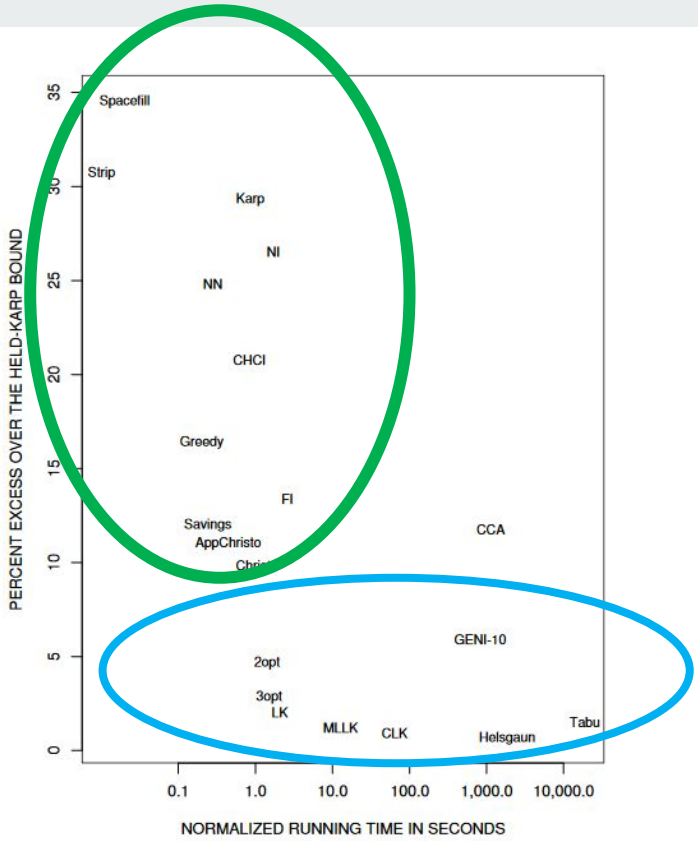
Algoritmos de Aproximación: Mejoramiento

ALGORITMOS DE MEJORA PARA EL PROBLEMA DEL VIAJANTE

- Algoritmo de Lin-Kernighan
- Estrategias de búsqueda tabú
- Estrategias de Recocido Simulado (Simulated annealing)
- Algoritmos genéticos

- Algoritmo de Arora: Garantiza un límite $(1+1/c)$ para cualquier $c>1$ en tiempo polinomial.
  Como vimos con la mochila, para valores de c muy grandes, el tiempo aún es excesivo.

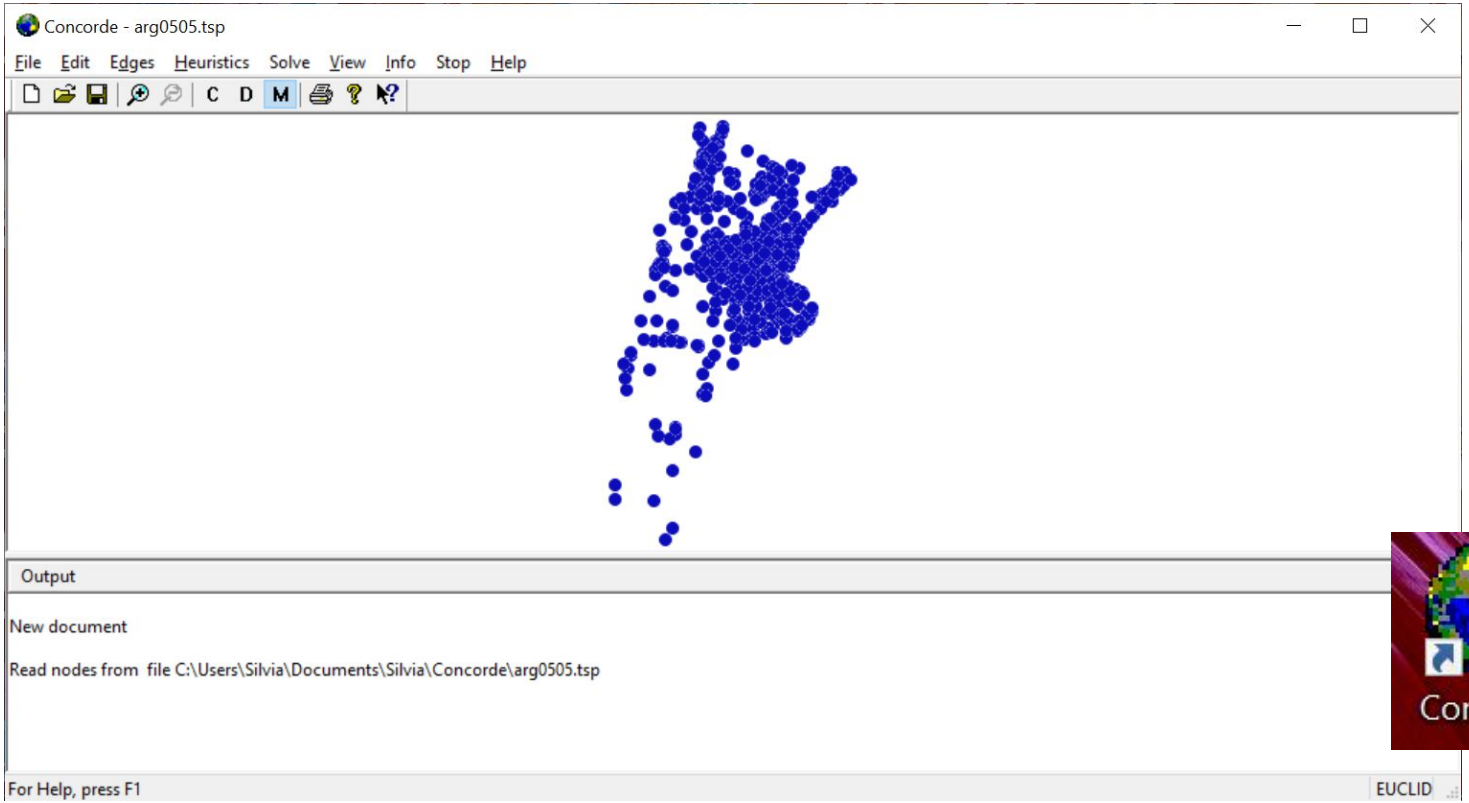
Algoritmos de Aproximación: Mejoramiento



CONSTRUCCIÓN

MEJORAMIENTO

Algoritmos de Aproximación





Algoritmos de aproximación: Conclusiones

- En algunos casos pseudopolinomiales, podemos obtener garantías de solución “casi” tan buenas como querramos, en tiempo polinomial.
  Por ej, en el problema de la mochila.
- Vimos diferentes estrategias para el problema del viajante de comercio.
- Diferencias entre algoritmos de construcción y algoritmos de mejora



¡Muchas gracias!